AI-Powered Company Research Assistant

Project Documentation & Submission Report

Manish Giri
Manishgiri8101@gmail.com

Built with Python, Flask, and Tailwind CSS

Date: July 4, 2026

1. Task Statement

Build an AI-powered Company Research Assistant that enables users to research any company by providing either the company name or its website URL. The application should automatically gather information from the company's website and other publicly available online sources, analyze the collected information using AI, identify potential competitors, and generate a professional downloadable PDF report.

The primary objective of this assignment is to evaluate the ability to design and build AI-powered applications that combine website crawling, AI reasoning, external APIs, and a modern user experience.

1.1 Deliverables Covered in This Document

- Public deployment URL for the live application
- Project README, local setup instructions, and environment variable documentation
- Description of the professional downloadable PDF report generated by the application, containing company and competitor information
- Complete technical documentation of the task: architecture, technology stack, workflow, and requirements

2. Project Overview

The AI-Powered Company Research Assistant is an intelligent web application designed to automate business research. Users input either a company name or a website URL, and the application searches the web, crawls the relevant website pages, uses AI to analyze the data, and produces a professional PDF report containing company details, products/services, target-audience pain points, and a competitor analysis.

The application follows a classic client-server architecture:

- **Frontend (Client):** A responsive, single-page chat-style application built with HTML, Tailwind CSS, and vanilla JavaScript, communicating with the backend via REST APIs.
- **Backend (Server):** A Python Flask application that orchestrates requests, integrates with third-party APIs (Serper, OpenRouter, Discord), performs web scraping, and generates PDF files.

3. System Architecture

The diagram below illustrates the end-to-end data flow between the frontend, the Flask backend orchestrator, and the external services used for search, crawling, AI synthesis, PDF generation, and Discord delivery.

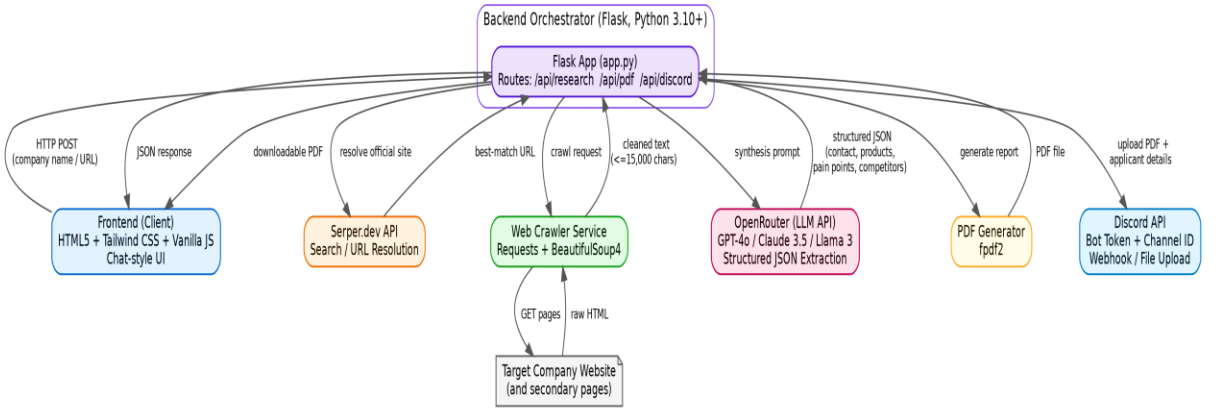

Figure 1: High-level component and data-flow architecture

3.1 Component Responsibilities

Component	Responsibility
Frontend (UI)	Collects user input (company name/URL), renders chat-style responses, triggers PDF download and Discord push.
Flask Backend	Central orchestrator; exposes /api/research, /api/pdf, and /api/discord endpoints; coordinates all downstream services.
Serper.dev API	Resolves the official company website from a name query; supplies supplementary search context.
Web Crawler (BeautifulSoup4 + Requests)	Fetches and parses HTML, strips boilerplate (scripts, styles, nav), extracts meaningful text, may crawl secondary pages.
OpenRouter (LLM API)	Synthesizes crawled text into structured JSON: company info, products, pain points, and competitors, using models such as GPT-4o, Claude 3.5, or Llama 3.
PDF Generator (fpdf2)	Builds a formatted, multi-page PDF report on demand from the structured JSON.
Discord API	Uploads the generated PDF and applicant details to a configured Discord channel via Bot Token/Webhook.

4. Technology Stack

4.1 Frontend

- HTML5 — semantic structure of the web application
- Tailwind CSS (via CDN) — rapid styling, responsive design, dark mode
- Vanilla JavaScript — DOM manipulation, state handling, and fetch-based API calls
- FontAwesome — scalable vector icons

4.2 Backend

- Python 3.10+ — core programming language
- Flask — lightweight web framework for routing and API endpoints
- BeautifulSoup4 & Requests — HTML scraping and text extraction
- fpdf2 — dynamic PDF report generation
- python-dotenv — secure environment variable management
- Gunicorn — WSGI HTTP server for production deployment

4.3 External APIs

- **Serper.dev:** Google Search proxy used to find official company websites and as a fallback for search snippets.
- **OpenRouter:** Unified AI API enabling dynamic switching between LLMs (GPT-4o, Claude 3.5 Sonnet, Llama 3) to analyze scraped text and generate structured JSON insights.
- **Discord API:** Pushes notifications and uploads generated PDF reports to a specified Discord channel using a Bot Token.

5. Key Features

- **Intelligent URL Resolution:** Uses Serper.dev to find the official website if only a company name is provided.
- **Automated Web Crawling:** Extracts clean, meaningful text from target websites while ignoring boilerplate and scripts.
- **AI Synthesis (OpenRouter):** Dynamically processes scraped data to extract contact info, products/services, pain points, and top competitors.
- **PDF Report Generation:** Instantly builds a formatted, multi-page, professional PDF report.
- **Discord Integration:** Automatically sends the generated PDF and applicant details to a configured Discord channel via Webhook/Bot API.
- **Modern UI:** ChatGPT-style animated chat interface with dark mode and dynamic loading states.

6. Execution Workflow

1. Input Handling: The user provides either a company name or a URL on the frontend.
2. Website Resolution (Serper): If a name is provided instead of a valid URL, the backend queries Serper.dev for "{Company Name} official website" to retrieve the highest-ranking official URL.
3. Web Crawling: The backend fetches the HTML with requests, parses the DOM with BeautifulSoup, strips scripts/styles/navigation, and extracts meaningful text — optionally crawling secondary pages such as /about or /products.

4. AI Analysis (OpenRouter): The extracted text (truncated to fit LLM context limits) is sent to OpenRouter via a strict system prompt instructing the model to extract company name, phone, address, services, pain points, and competitors as a JSON object.
5. Report Generation: The JSON is displayed in the chat interface; on request, the /api/pdf endpoint builds a multi-page PDF using fpdf2.
6. Discord Integration: If configured, the backend uploads the PDF and a formatted message with applicant/company details to Discord via the Bot API.

7. Requirements

7.1 Functional Requirements

1. Input Flexibility: Must accept both company names (e.g., "Microsoft") and URLs (e.g., "https://microsoft.com").
2. Search Capabilities: Must use Serper.dev for finding official URLs and gathering supplementary context.
3. Web Crawling: Must extract meaningful textual content from the target company's web pages.
4. AI Processing: Must utilize OpenRouter to allow dynamic AI model selection (e.g., GPT-4o, Claude 3.5) for text analysis.
5. Data Extraction: Must successfully identify and extract Company Name, Website, Phone, Address, Products, Pain Points, and Competitors.
6. Export: Must generate a downloadable PDF report summarizing the findings.
7. Discord Integration: Must allow users to input a Bot Token and Channel ID to automatically forward the generated PDF.

7.2 Non-Functional Requirements

- **Performance:** The crawling and AI synthesis pipeline should aim to complete within 15–30 seconds.
- **Usability:** The UI must be intuitive, responsive (mobile/desktop), and show loading indicators for background processes.
- **Maintainability:** Code must be modular, separated into services/, templates/, and static/.
- **Security:** API keys must never be exposed client-side and must be managed via environment variables on the backend.
- **Context Limits:** Crawled text is limited to 15,000 characters to prevent LLM token-limit errors.

8. Project Structure

```
relu_consultancy/
│
├── app.py                      # Main Flask application entry point
├── requirements.txt            # Python dependencies
├── .env                        # Environment variables (excluded from Git)
```

```
│
├── docs/                       # Project documentation
│   ├── Project_Architecture.md
│   └── Deployment_Guide.md
│
├── services/                   # Backend API services
│   ├── crawler.py              # BeautifulSoup web scraper
│   ├── discord.py              # Discord API integration
│   ├── openrouter.py           # LLM processing logic
│   ├── pdf_generator.py        # fpdf2 PDF creation
│   └── serper.py               # Google search proxy logic
│
├── static/                     # Frontend assets
│   ├── css/styles.css
│   └── js/app.js
│
└── templates/
    └── index.html              # Main UI layout (Tailwind CSS)
```

9. Local Setup Instructions

9.1 Prerequisites

- Python 3.10 or higher installed.

9.2 Installation

Clone the repository (or download the files) and navigate to the project directory:

```
cd relu_consultancy
```

Create a virtual environment:

```
# On Windows
python -m venv venv
.\venv\Scripts\activate

# On Mac/Linux
python3 -m venv venv
source venv/bin/activate
```

Install the dependencies:

```
pip install -r requirements.txt
```

9.3 Environment Variables

Create a file named .env in the root directory and add the following API keys:

```
SERPER_API_KEY="my_serper_api_key"
OPENROUTER_API_KEY="my_openrouter_api_key"
```

Variable	Provider	Purpose	Get a Key
SERPER_API_KEY	Serper.dev	Resolves official company websites from a name query and provides search snippets as a fallback.	https://serper.dev/
OPENROUTER_API_KEY	OpenRouter	Authenticates LLM calls used to synthesize crawled text into structured company/competitor data.	https://openrouter.ai/

Both keys are free-tier available. Keys must never be committed to Git or exposed to the client-side browser; they are read only by the Flask backend via python-dotenv.

9.4 Running the Application

Start the Flask development server:

```
python app.py
```

Then open a web browser and navigate to:

```
http://127.0.0.1:5000
```

10. API Token Collection Process

10.A Serper API Key

7. Navigate to serper.dev.
8. Create a free account.
9. From the dashboard, copy the generated API key.

10.B OpenRouter API Key

10. Navigate to openrouter.ai.
11. Create a free account.
12. Navigate to Keys (Profile → Keys).
13. Click Create Key, name it, and copy the secret token immediately.

11. Deployment

The application is production-ready and configured for deployment on platforms such as Render.com using the included gunicorn dependency.

11.1 Public Deployment URL

Live application URL: *[https://company-research-assistant-z4mw.onrender.com]*

11.2 Phase 1: Uploading to GitHub

Because this app uses sensitive API keys, the .env file and the venv (virtual environment) folder must never be uploaded. A .gitignore file is provided to prevent this automatically.

8. Go to GitHub.com and log in.
9. Click the "+" icon in the top right corner and select "New repository".
10. Name the repository (e.g., company-research-assistant) and set it to Public.
11. Check the box "Add a README file".
12. Click "Create repository".
13. On the new repository page, click "Add file" → "Upload files".
14. Drag and drop services/, static/, templates/, docs/, app.py, requirements.txt, and README.md into the browser.
15. Click "Commit changes" at the bottom of the page.

11.3 Phase 2: Deploying to Render.com

16. Log in to Render.com.
17. From the Dashboard, click "New +" and select "Web Service".
18. Select "Build and deploy from a Git repository" and click Next.
19. Search for and select the repository created in Phase 1.
20. Configure the deployment: Runtime = Python 3; Build Command = pip install -r requirements.txt; Start Command = gunicorn app:app; Instance Type = Free.
21. Under Advanced, add environment variables SERPER_API_KEY and OPENROUTER_API_KEY with the actual key values.
22. Click "Create Web Service".
23. Within 2–4 minutes, Render builds the environment and provides a public URL for the live application — record it in Section 11.1 above.

12. Generated PDF Report — Company & Competitor Information

In addition to the in-chat display of results, the application produces a professional, downloadable, multi-page PDF report via the /api/pdf endpoint. The report is generated on the fly using fpdf2 from the structured JSON returned by the AI synthesis step, and includes:

- **Company Overview:** resolved company name, website, and a summary description.
- **Contact Information:** phone number and physical address, where available on the source site.
- **Products & Services:** a synthesized summary of what the company offers.
- **Target Audience Pain Points:** problems the company's offering is positioned to solve.

- **Competitor Analysis:** a list of top competitors identified by the LLM from the crawled content and search context.

Once generated, the same PDF can optionally be pushed automatically to a configured Discord channel, along with the applicant's details, via the Discord Bot API.

13. Tasks Achieved

- Project Scaffold & Initialization: environment setup using Python and Flask.
- Search Integration: used Serper.dev to find official company websites from text inputs.
- Web Crawling Engine: built a custom BeautifulSoup4 scraper that intelligently extracts text while ignoring boilerplate HTML.
- AI Synthesis: integrated OpenRouter to process scraped text and generate structured JSON insights dynamically.
- PDF Generation: implemented on-the-fly PDF creation using fpdf2.
- Discord Webhook: added the ability to push messages and PDF files directly to Discord.
- Frontend UI/UX: built a responsive, animated chat UI using HTML, Vanilla JS, and Tailwind CSS.

14. Conclusion

The AI-Powered Company Research Assistant satisfies the assignment's objective of combining website crawling, AI reasoning, external APIs, and a modern user experience into a single cohesive application. It resolves a company's identity from minimal input, gathers and cleans real website content, uses an LLM to reason over that content and identify competitors, and packages the findings into a professional PDF deliverable — with optional automatic distribution via Discord. The codebase is modular, environment-variable driven for security, and ready for free-tier deployment on Render.com.